

Technical Grounding Report: Zero-Budget Online Auction Marketplace for India

Prepared as source material for an implementation plan and Claude Code build prompt. All free-tier figures verified as of September 2026; verify on official pricing pages before building, as these change frequently.

TL;DR

- **Build it as a single Next.js 15 full-stack app on Vercel + Supabase (Postgres, Auth, Storage, Realtime), with auction closure driven by Upstash QStash delayed messages, not cron** — this plays to the developer's existing Next.js/Supabase strengths, avoids the cross-origin httpOnly-cookie pain of a split Vercel+Render deployment, and closes auctions at the exact second they expire (which no free cron can do). Add a small FastAPI service only as an on-demand job for the future image-moderation ML.
- **Two hard legal red lines: (1) never collect or store real Aadhaar numbers** — even in a demo it is legally hazardous under the Aadhaar Act (public-display/storage prohibitions, penalties up to ₹1 crore), and the *Puttaswamy* Supreme Court judgment of 26 September 2018 struck down the part of Section 57 that let private entities do Aadhaar authentication; and **(2) never actually hold buyer funds** — pooling money before paying sellers requires an RBI Payment Aggregator licence (₹15 crore net worth at application, rising to ₹25 crore). Both are trivially avoided by keeping KYC and payments simulated, which is exactly what the students want.
- **The demo is fully achievable at ₹0**, but the "must stay awake" requirement fights the free tier: Supabase pauses after 7 days idle (returns HTTP 540) and Render free web services sleep after 15 min. Use client-side Verhoeff/PAN format validation for fake KYC, Razorpay **test mode** (free, no company docs) for fake payments, and a persistent demo banner. The GitHub Student Developer Pack materially raises the effective budget (Azure \$100 credit, free domain + SSL, Sentry, Heroku \$13/mo×24) — but verify offers at redemption since DigitalOcean's \$200 credit ended Aug 1, 2026.

Key Findings

1. Recommended architecture

A single Next.js 15 app (App Router, Route Handlers + Server Actions) deployed on Vercel Hobby, backed by Supabase (Postgres + Auth + Storage + Realtime). Auction expiry handled by QStash scheduled/delayed HTTP callbacks. This is the lowest-friction, lowest-cost path and eliminates the cross-origin httpOnly-cookie problems the developer already hit with Vercel↔Render.

2. Free-tier reality check (verified Sept 2026)

- **Supabase Free:** 500 MB Postgres, 1 GB storage, 5 GB egress, 50K MAU, 200 concurrent Realtime connections, 2M Realtime messages, 500K Edge Function calls, 2 projects, 60 direct / 200 pooler DB connections. Supabase's own docs confirm free-plan projects are

paused after a 7-day low-activity period (paused projects return HTTP 540; paid projects cannot be paused). [UI Bakery](#) **Zero backups on free**. This pause is the single biggest threat to a "live" demo — mitigate with a GitHub Actions scheduled ping every 3 days (GitHub integration is free on all plans as of April 2026).

- **Neon Free:** 0.5 GB storage/project, 100 CU-hours/month (doubled from 50 in Oct 2025 post-Databricks acquisition), scale-to-zero with ~500 ms cold wake, up to 100 projects, 10 branches. Good Postgres alternative but scale-to-zero adds first-query latency; storage cap is tighter than Supabase.
- **Render Free:** Render's docs confirm web services **spin down after 15 minutes without inbound traffic and take about one minute to spin back up**; 750 instance-hours/month, 512 MB RAM / 0.1 CPU. Free Postgres is 1 GB and **expires 30 days after creation** (14-day grace) — disqualifying for a persistent demo DB. Render cron jobs are a **paid** feature (\$1/mo minimum, billed by the second).
- **Vercel Hobby:** functions default 30s; with Fluid Compute, up to 5 min on free. [Vercel](#) **Cron on Hobby = once-per-day minimum, timing only guaranteed within the hour, UTC only, ~2 cron jobs/project** — useless for closing auctions that end at an arbitrary second. Any external scheduler can call the same API route, since only Vercel's built-in scheduler is rate-limited. [Runhooks](#)
- **Real-time:** Supabase Realtime (200 concurrent connections free) is the natural choice. Pusher Sandbox = 200 concurrent connections + 200K messages/day (daily cap). [PkgPulse](#) Aply free = 6M messages/month (monthly cap) with stronger ordering/history guarantees. Self-hosted Socket.IO on Render is undermined by the 15-min spin-down, so managed real-time is the only viable free path on serverless hosting.
- **Storage/images:** Supabase Storage (1 GB, 50 MB max file, bundled) is simplest; Cloudflare R2 (10 GB, 1M Class A + 10M Class B ops/month, **zero egress**) [LeanOps Technologies](#) is best value and stable; Cloudinary Free = 25 credits/month (1 credit = 1 GB storage OR 1 GB bandwidth OR 1K transforms) [Agentdeals](#) and adds on-the-fly image transforms — useful for thumbnails.
- **Background jobs:** **Upstash QStash free = 1,000 messages/day** (raised from 500 per Upstash's GA announcement), 50 GB bandwidth, 1 MB message size, delayed delivery up to 1 year, cron, automatic retries with exponential backoff, and a dead-letter queue. [Upstash](#) This is the recommended auction-closure mechanism. Alternatives: Inngest free (50K runs/month, durable step functions), Supabase pg_cron (minute granularity), GitHub Actions scheduled (5-min minimum, drifts).
- **Email:** Resend/Brevo/Mailgun/SendGrid/Amazon SES all have free tiers; Mailgun via Student Pack = 20K emails/month for 12 months. (Exact current daily/monthly caps were not verified first-party in this pass — confirm before relying on specific numbers.)

3. Auction concurrency (the core correctness problem)

The canonical fix for simultaneous bids is a **Postgres transaction with** `SELECT ... FOR UPDATE` row lock on the auction row, or a single conditional `UPDATE auctions SET current_price = :new WHERE id = :id AND current_price < :new` (optimistic — the first of two concurrent bids wins, the second's WHERE fails). PostgreSQL's decision tree: row lock when you need to lock

specific rows and can wait; `(FOR UPDATE SKIP LOCKED)/(NOWAIT)` when you can't; advisory locks for application-level coordination; a `(version)` column for optimistic concurrency; `(SERIALIZABLE)` isolation when conflicts must be detected without explicit locks. `(SELECT FOR UPDATE)` trades latency for correctness — the right default for money-like operations.

The open-source `(kavinmd/Auction)` (FastAPI+Postgres+React) implements exactly this: a `(SELECT FOR UPDATE)` transaction that validates status/end-time/amount, then inserts the bid and updates price atomically, verified by an automated concurrency test using `(asyncio.gather())`. **Make closure idempotent** (check `(status)` before settling) so a retried QStash message, a duplicate cron fire, or a missed window can never double-close or double-charge.

4. Proxy bidding vs simple highest-bid

eBay uses **proxy (automatic) bidding**: each bidder enters a hidden maximum; the system bids only the minimum increment needed to keep them winning, so the final price = second-highest bid + one bid increment (up to the leader's max). Increments scale with price band. This is why "last-second snipes" and proxy bids coexist — the sniper's single max bid is compared against the current leader's hidden max. **None of the studied open-source repos implement true proxy bidding**, so the student must build it. The user's stated design is simple highest-bid, which is fine for an MVP; proxy bidding + soft-close anti-sniping is the strongest "production-ready" upgrade to propose.

5. India legal red lines (must-not-do list)

- **Aadhaar**: Aadhaar Act Section 29(4) prohibits publicly displaying/publishing any Aadhaar number; the only lawful use is a consented yes/no authentication or e-KYC via UIDAI. Section 40 penalises using collected identity information for any other purpose (imprisonment up to 3 years). Proposed amendments add civil penalties up to ₹1 crore per contravention (plus up to ₹10 lakh/day continuing). Critically, in **Justice K.S. Puttaswamy v. Union of India (26 September 2018)**, the Supreme Court by 4:1 majority (para 219(e)) held the portion of Section 57 enabling body corporates/individuals to seek Aadhaar authentication unconstitutional as violative of privacy. **Conclusion: do not collect or store real Aadhaar numbers, even in a demo.** Offline routes (DigiLocker, Aadhaar XML, masked Aadhaar) exist for a real product but are out of scope for a demo.
- **DPDP Act 2023**: Rules notified 13 Nov 2025; the Data Protection Board is established; **substantive obligations and penalties commence ~14 May 2027** (18 months after notification; the IT Secretary has said there will be no grace period from Day 1). Per PIB, the highest penalty is **up to ₹250 crore** for failure to maintain reasonable security safeguards, ₹200 crore for breach-notification/children's-data failures, ₹50 crore for other violations. Applies to any digital personal-data processing in India, regardless of company size. Practise data minimisation now — another reason not to store identity documents.
- **RBI Payment Aggregator**: Per RBI Guidelines DPSS.CO.PD No.1164/02.14.008/2019-20, a PA must have **minimum net worth ₹15 crore at application, rising to ₹25 crore by the**

end of the third financial year and maintained thereafter, plus a scheduled-bank escrow/nodal account, PCI-DSS, and data localisation. Handling/pooling merchant funds without authorisation is illegal under the Payment and Settlement Systems Act, 2007. Real marketplaces therefore never hold funds directly — they integrate a **licensed PA's marketplace split product** (Razorpay Route or Cashfree Easy Split, both of which onboard vendors, run vendor KYC — bank/UPI/PAN verification — and auto-split settlements). **A student demo must never touch real money; use test mode only.**

- **Consumer Protection (E-Commerce) Rules 2020** (Notification GSR 462(E), 23.07.2020): a marketplace entity must appoint a **grievance officer who, per Rule 4(4), acknowledges every consumer complaint within 48 hours and redresses it within one month**, display its legal name/office address/website and customer-care and grievance-officer contact details, issue complaint ticket numbers, and — importantly — **ensure sellers disclose their identity, address, and contact details**. This mandatory seller disclosure **directly conflicts with the user's buyer-seller anonymity goal** and is the single biggest legal tension in the design; flag it explicitly and treat anonymity as a demo-only feature or a masked-relay pattern rather than true non-disclosure for a real product.
- **IT Act intermediary liability / IT Rules 2021**: safe harbour under Section 79 requires due diligence, published grievance mechanisms, and timely takedown of unlawful content — reinforcing the need for the admin moderation queue.
- **Prohibited goods** (for the moderation checklist): weapons/ammunition; explosives and hazardous chemicals (the CCPA issued notices to 8 platforms including IndiaMART and Justdial in May 2026 over online listings of ammonium nitrate, gunpowder, picric acid, PETN); narcotics (NDPS Act 1985); wildlife products (Wildlife Protection Act 1972 / CITES); counterfeit and IP-infringing goods; tobacco and **e-cigarettes/vaping (banned nationwide since 2019)**; prescription drugs and medical devices; antiquities (Antiquities and Art Treasures Act 1972); maps misrepresenting India's borders.
- **GST**: marketplace operators face **TCS under GST** and **Section 194-O TDS** obligations on seller payouts for a real product — out of scope for a demo but relevant to any production plan.

6. Demo-safe fake KYC & payments

- **Fake KYC**: implement **client-side format validation only** — PAN regex `[A-Z]{5}[0-9]{4}[A-Z]` (case-normalise first) and Aadhaar 12-digit + **Verhoeff checksum** using the standard multiplication (`(d)`), permutation (`(p)`), and inverse (`(inv)`) lookup tables. Note: UIDAI never issues an Aadhaar starting with 0 or 1 (reserved for test/internal numbers), and a random 12-digit number passes Verhoeff only ~10% of the time, so the check "feels real." Because full verification requires UIDAI's OTP-consented API, format validation collects and verifies **no real identity data** — the correct demo posture. Use obviously-synthetic values (e.g., the known synthetic fixture `234567890124`), which passes Verhoeff but is not an issued Aadhaar).
- **Fake payments: Razorpay test mode** — free, no company documents required, generates separate test API keys; simulate UPI success with `success@razorpay`, and use Razorpay's published test card numbers with any future expiry. RazorpayX test mode

has its own dummy balance for simulating payouts. Cashfree sandbox and Stripe test mode are equivalent alternatives. Keep a **persistent "DEMO — no real money or identity is processed" banner** so it cannot be mistaken for a live money-handling service.

7. Disintermediation prevention

Real marketplaces prevent off-platform deals with layered controls: **email aliasing** (eBay routes through addresses like `...@members.ebay.com` so the seller never sees the buyer's real email); **message scanning** that rejects/blocks messages containing phone numbers, emails, or URLs (eBay explicitly prohibits pre-transaction contact-info sharing); **ML/NLP detection** including OCR on images and PDFs (Fiverr); **temporary masked phone numbers with expiry** (Airbnb issues temp numbers active from booking until ~2 days after checkout, per its help article; Uber India has used disguised relay numbers since 2015); and **behavioral/device fingerprinting** (IP, device, typing patterns) to catch linked evasion. Upwork and Fiverr both make circumvention a **permanent-ban** offense.

Documented failure modes the student should expect: **obfuscation defeats regex** (spelling "at"/"dot," or tools like SaniText that insert symbols into banned words); **false positives** silently block legitimate post-sale logistics messages (eBay sellers often get no notice); and **the shipping label is the hardest leak** — physical fulfilment must expose at least one address unless the platform generates the label itself, inserts an intermediary hub/authenticator (as eBay does for Authenticity Guarantee items using virtual tracking numbers), or uses pickup points. **Design implication:** for the anonymity requirement, combine in-app-only chat + regex/ML PII scrubbing + platform-generated shipping labels / masked relay contact — and treat pure message filtering as necessary but insufficient.

8. Open-source references worth studying

- [kavinmd/Auction](#) (AuctionSphere) — FastAPI + PostgreSQL + React (Vite). Strongest concurrency model: `SELECT ... FOR UPDATE` transaction validated by an `asyncio.gather` concurrency test; **implements anti-sniping** (bid in last 60s auto-extends 2 min); WebSocket live updates, Stripe, auto-close scheduling. **Verify LICENSE file before reuse.** Closest match to the recommended stack for a Python backend.
- [theavnishkumar/online-auction-system](#) — MERN (React 19, Node/Express 5, MongoDB, Socket.io), MIT, ~42★. Best-documented (ARCHITECTURE.md, LEARNING_GUIDE.md); real-time bidding via Socket.io rooms; race-condition prevention via atomic `findOneAndUpdate` with a price condition; httpOnly-cookie JWT. No proxy bidding/anti-sniping.
- [hazemAzzam/Online_Auction](#) — Django REST, MIT. Clean relational schema: buyer/seller roles, unique item numbers, fixed category hierarchy, bids, highest-bid resolution, post-transaction feedback/ratings. [GitHub](#) Good schema reference.
- [hmellor/auction-website](#) — GitHub Pages + Firebase (anonymous auth + Realtime Database). Zero-budget realtime reference with security rules preventing modification/deletion of existing bids. [GitHub](#) **Verify LICENSE.**
- [JarrodMalkovic/auction-website](#) — Node microservices + Kubernetes + Stripe. Event-driven scaling reference (overkill for MVP). **Verify LICENSE.**

None implement true **proxy/max-bid auto-bidding** — that remains a build item.

9. GitHub Student Developer Pack (raises the effective budget above ₹0)

Authoritative list: (education.github.com/pack) and the (github-education-resources/Student-Developer-Pack-Current-Partners-FAQ) repo (partner table self-dated Oct 2024 — **re-verify each offer at redemption**). Current, India-eligible highlights:

- **Microsoft Azure for Students: \$100 credit + free access to 25+ services** (App Service, Functions, MySQL in-app), no credit card, **age 18+** — now the primary surviving cloud-credit option.
- **Heroku: \$13/month credit for 24 months** — enough for a small always-on dyno + hobby Postgres.
- **MongoDB Atlas \$50; Namecheap free (.me) + 1-year SSL; Name.com free 1-year domain + security (SSL + WHOIS privacy); .TECH free 1-year domain.**
- **Sentry** (50K errors, 100K transactions, 1 yr), **New Relic** (free while a student), **Datadog Pro** (2 yrs); **GitHub Copilot + Pro free**; **Mailgun 20K emails/month for 12 months** (bid notifications); **Stripe** first \$1,000 processing fees waived.
- **DigitalOcean's \$200 credit has ended — last redemption 31 July 2026, all credits retired 1 Aug 2026.** ([AI Student Discount](#)) **Do not rely on it.**
- Indian students verify via **student ID/enrollment letter/transcript upload** (most institutions lack a (.edu) email); enable 2FA, match profile name to documents; validation lasts 2 years.

Details

Architecture decision: monolith Next.js vs split Next.js+FastAPI

Given the developer already hit cross-origin httpOnly-cookie problems between Vercel and Render, and that free real-time and background-job support is cleanest inside the Supabase/Vercel ecosystem, **option (a) — a single Next.js 15 app + Supabase — is recommended for the MVP.** Auth via Supabase Auth (`auth.uid()` drives Row-Level Security), Postgres for all data, Supabase Realtime for live bids, Supabase Storage or Cloudflare R2 for images, QStash for closure. A **separate FastAPI service (option b)** is justified only for the future ML image-moderation "future scope," where Python's scikit-learn/Hugging Face ecosystem lives — and even then it should run as an **on-demand job**, not an always-on service, to sidestep Render's spin-down. The developer's FastAPI fluency makes this a low-risk add-on later, but shipping it in the split-deployment cookie configuration for auth is the pattern to avoid.

The "stay awake" problem

Two independent idling behaviours threaten the demo: Supabase's 7-day pause and (if used) Render's 15-min web-service spin-down. The Next.js+Supabase+Vercel path removes the Render spin-down entirely (Vercel functions are serverless — no persistent process to sleep) and reduces the problem to keeping Supabase warm via a scheduled ping (GitHub Actions cron on a public repo, or QStash) every 3–5 days. If the demo must be truly always-on with backups, Supabase Pro (\$25/mo) removes the pause — but that breaks the ₹0 constraint, so the

ping workaround is the recommended free path.

Auction closure design

Because auctions end at an arbitrary second and free cron granularity is coarse (Vercel Hobby = daily; pg_cron/GitHub Actions = minute), the robust pattern is: **when an auction goes live, publish a QStash message with a delay equal to the auction duration**, targeting an idempotent closure endpoint. On any soft-close or 48-hour extension, publish a new delayed message. The closure handler must guard on auction (`status`). Keep a periodic sweep (pg_cron or GitHub Actions every few minutes) as a **safety net** to catch any auction whose delayed message was lost — this makes the system resilient to a crashed process or missed callback.

Real-time bids and countdown timers

Use Supabase Realtime (Postgres change subscriptions) to broadcast new highest bids; on serverless free hosting this is preferable to self-hosted WebSockets. The developer's existing React polling hook is a valid fallback if Realtime connection caps (200 concurrent) are approached. Countdown timers must be **server-authoritative**: the server returns (`ends_at`) (UTC) and the client renders the countdown, periodically re-syncing to correct clock skew; never trust the client clock to decide closure.

Audit trail

Store an **append-only** (`bids`) table (never update/delete rows) as the immutable bid history for dispute resolution; derive (`current_price`) from it or maintain a locked denormalised column updated in the same transaction. This also supports fraud detection.

Known fraud patterns to design against

Shill bidding (seller/associates inflating price — detect via seller-buyer account links, IP/device overlap), bid shielding (fake high bid retracted to let an associate win low), non-paying bidders (the user's strike/ban rule addresses this), seller collusion, and fake listings (the admin approval queue addresses this). Behavioral/device fingerprinting is the documented mitigation.

User's specific rules mapped to data model & mechanics

- **1-hour to 1-month window** → store (`ends_at`); schedule QStash accordingly.
- **Winner pays within a day or account suspended; 2 strikes = permanent ban** → model (`payment_deadline`), a (`strikes`) counter, and account (`status`); enforce via a scheduled check + QStash delayed message at the deadline.
- **Second-highest fallback + seller choice + 48-hour extension** → model a seller-offer state; on seller rejection, extend (`ends_at`) by 48h and reschedule closure (a custom soft-close variant).
- **Admin approval of listings and of suspicious prices** → moderation queue with (`status`) (pending/approved/rejected) gating public visibility; the "laptop for 10 crore" case is a price-sanity flag in this queue.
- **Anonymity** → no contact fields exposed anywhere; in-app messaging only, with regex/ML PII scrubbing; shipping via platform-generated label pattern (simulated in the demo).

- **View without login; login only to act** → public read routes for listings; gate bid/sell/comment/like behind Supabase Auth.

Recommendations

1. **Stage 1 — MVP at ₹0:** Next.js 15 + Supabase (Auth/DB/Storage/Realtime) on Vercel Hobby. Fake KYC = client-side PAN regex + Aadhaar Verhoeff format check with synthetic data (**NO real Aadhaar collection or storage**). Fake payments = Razorpay test mode. Auction closure = QStash delayed messages + idempotent handler + pg_cron/GitHub Actions safety sweep. Keep Supabase warm with a scheduled ping. Admin moderation = Supabase Studio first, then a custom Next.js `/admin` route with RLS-gated access. Append-only bids table from day one.
2. **Stage 2 — portfolio polish:** add proxy bidding, soft-close anti-sniping, in-app messaging with regex PII scrubbing, a full audit/dispute view, and the persistent "DEMO" banner. Add Sentry (Student Pack) for error tracking.
3. **Stage 3 — toward production (only if serious):** get legal advice on the E-Commerce Rules grievance-officer and **seller-disclosure vs anonymity conflict**; integrate a licensed PA (Razorpay Route / Cashfree Easy Split) rather than holding funds; replace raw Aadhaar with offline DigiLocker verification; complete DPDP data-minimisation and breach-notification readiness before 14 May 2027; register GST/TCS obligations.
4. **Budget lever:** activate the GitHub Student Pack early for the Azure \$100 credit, a free domain + SSL, Mailgun email, and Sentry — this is the cheapest way to move above the raw free tier without spending.

Thresholds that change the recommendation:

- Concurrent live bidders approaching **200** (Supabase Realtime cap) or **200K daily real-time messages** → move real-time to Ably (6M/month free) or upgrade Supabase.
- DB approaching **500 MB**, or the pause becoming unacceptable for judging/demo day → Supabase Pro (\$25/mo) or use Student Pack Azure credit for a hosted Postgres.
- QStash usage approaching **1,000 messages/day** → batch closures or add pg_cron sweeps as the primary mechanism.
- Any intent to handle **real money or real identity** → stop and get legal/compliance advice first (PA licence, Aadhaar, DPDP).

Caveats

- Free-tier figures shift frequently; multiple sources explicitly note Supabase/Neon limits "have changed several times." Verify on official pricing pages before architecting around a specific cap.
- The enricher corrected the QStash free tier to **1,000 messages/day** (first-party Upstash), resolving a 500-vs-1,000 discrepancy in secondary sources.
- Supabase's 200-concurrent-Realtime figure comes from secondary aggregators consistent with the pricing page; confirm first-party if it is load-bearing.
- Fiverr's "~99% NLP catch rate" and image-OCR claims come from third-party SEO/

recovery blogs, not Fiverr; treat as indicative only.

- GitHub Student Pack partner values are partly stale (table dated Oct 2024); the DigitalOcean credit is confirmed dead, and other dollar figures should be re-checked at redemption.
- Transactional-email daily/monthly caps and free image-moderation quotas (Google Cloud Vision SafeSearch, AWS Rekognition free tier, Hugging Face NSFW/CLIP models) were not verified first-party in this pass — check current limits before committing. For the "future scope" AI flagging, realistically-free options are open Hugging Face models (NSFW detectors, CLIP zero-shot) run in an on-demand FastAPI job; hosted-cloud free vision quotas are small and time-limited.
- The buyer-seller **anonymity requirement legally conflicts with the E-Commerce Rules' mandatory seller disclosure** — acceptable for a demo, but must be resolved (masked relay + disclosed-to-platform) before any real launch.